

OpenStack Self-Scaling Infrastructure — Build Log

macOS Monterey · VirtualBox · Ubuntu 24.04 · DevStack · Ceilometer / Gnocchi / Aodh / Heat

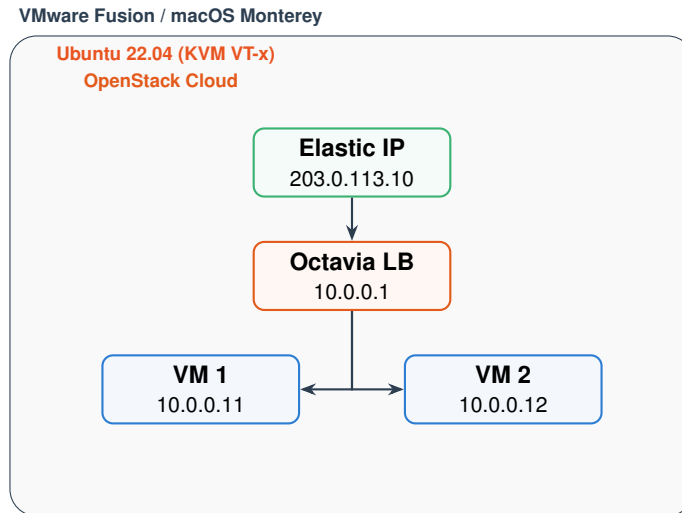


1. Discover OpenStack

- **Goal:** Deploy OpenStack locally on Apple Intel hardware via DevStack.
- **Constraint:** OpenStack Nova requires KVM. macOS intercepts VT-x natively.
- **Solution:** Establish a nested virtualization chain to bypass host restrictions and pass hardware acceleration to the VMs.

2. Initial Concept — Static Load-Balanced Pair

The initial architecture established a static pair of application servers behind a load balancer, all nested inside the OpenStack environment.



Note: this Octavia LBaaS branch was later abandoned in favour of a manual HAProxy VM. Octavia setup proved unreliable in DevStack without significant extra configuration.

3. Research OpenStack Components

- **Nova** — Compute service managing VM lifecycles via the KVM hypervisor.
- **Keystone** — Identity service handling authentication, authorization, and trusts.
- **Ceilometer** — Telemetry agent; polls hypervisor/Nova for resource usage and publishes samples.
- **Gnocchi** — Time-series metric database; stores samples under per-resource archive policies.
- **Aodh** — Alarm evaluation service; queries Gnocchi and fires actions when a threshold is crossed.
- **Heat** — Orchestration engine; reads HOT templates and creates/destroys resources in response to signals.

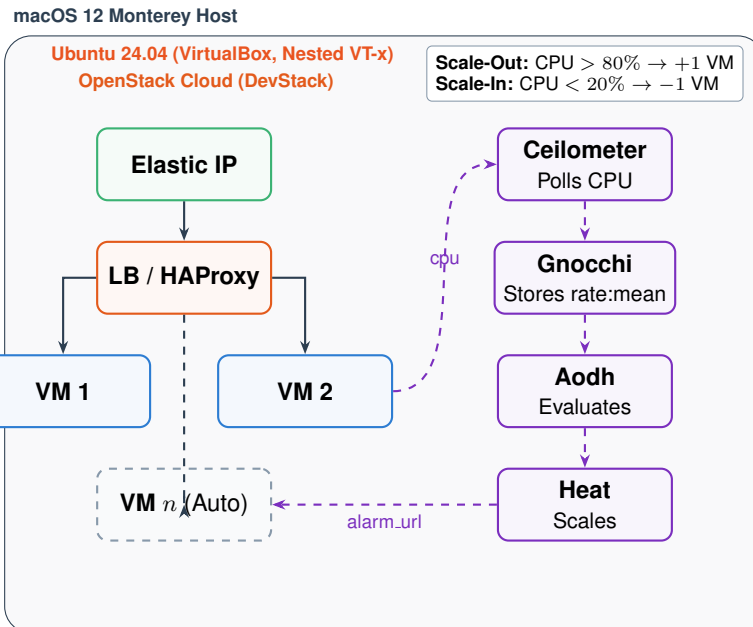
4. Environment Setup

Migrated to Ubuntu 24.04 LTS via VirtualBox to comply with DevStack's modern Python requirements.

1. Forced nested KVM VT-x: `VBoxManage modifyvm "openStack" --nested-hw-virt on`
2. Bridged VirtualBox networking to the host to avoid MAC filtering drops.
3. Cloned DevStack, configured `local.conf`, executed `./stack.sh`.

5. Target Architecture

The static pair was replaced by a dynamic pool managed by a closed telemetry/orchestration loop, scaling strictly on CPU utilization.



6. The Bugs Encountered

Bug — signal_url vs alarm_url (root cause of all 401s)

BUG: Every alarm action returned 401 Unauthorized

OS::Heat::ScalingPolicy exposes two different attributes:

- signal_url — requires a valid Keystone token.
- alarm_url — pre-signed, no token required.

The template used signal_url throughout. Aodh's notifier correctly fired on every threshold breach but every HTTP POST bounced with 401 because Aodh does not attach a bearer token to webhook calls. The stack's trust and Aodh's service-user credentials both verified correct independently, making this bug resistant to diagnosis.

FIX: Swap the attribute

```
alarm_actions:
  - { get_attr: [scale_out_policy, alarm_url] } # not signal_url
```

Confirmed via openstack orchestration resource type show OS::Heat::ScalingPolicy. After the swap, manual `curl -X POST $ALARM_URL` returned 200 OK with no Authorization header, and Aodh-triggered scale-outs began provisioning VMs automatically.

7. Final Working Heat Template

```
heat_template_version: 2021-04-16
description: Auto-scaling CirrOS instances on CPU thresholds (validated working config).

resources:

  web_server_group:
    type: OS::Heat::AutoScalingGroup
    properties:
      min_size: 1
      max_size: 3
      resource:
        type: OS::Nova::Server
        properties:
          flavor: m1.tiny
          image: cirros-0.6.3-x86_64-disk
          networks:
            - network: private
          metadata:
            server_group: { get_param: "OS::stack_id" }

  scale_out_policy:
    type: OS::Heat::ScalingPolicy
    properties:
      adjustment_type: change_in_capacity
      auto_scaling_group_id: { get_resource: web_server_group }
      scaling_adjustment: 1
      cooldown: 120

  scale_in_policy:
    type: OS::Heat::ScalingPolicy
    properties:
      adjustment_type: change_in_capacity
      auto_scaling_group_id: { get_resource: web_server_group }
      scaling_adjustment: -1
      cooldown: 180

  cpu_alarm_high:
    type: OS::Aodh::GnocchiAggregationByResourcesAlarm
    properties:
      description: "Scale OUT - CPU > 80%"
      metric: cpu
      aggregation_method: rate:mean
      granularity: 300
      evaluation_periods: 1
      threshold: 800000000
      comparison_operator: gt
      resource_type: instance
      query:
        str_replace:
          template: '{"=": {"server_group": "STACK_ID"}}'
          params:
            STACK_ID: { get_param: "OS::stack_id" }
      alarm_actions:
        - { get_attr: [scale_out_policy, alarm_url] }

  cpu_alarm_low:
    type: OS::Aodh::GnocchiAggregationByResourcesAlarm
    properties:
      description: "Scale IN - CPU < 20%"
      metric: cpu
      aggregation_method: rate:mean
      granularity: 300
      evaluation_periods: 1
      threshold: 200000000
      comparison_operator: lt
      resource_type: instance
      query:
        str_replace:
          template: '{"=": {"server_group": "STACK_ID"}}'
          params:
            STACK_ID: { get_param: "OS::stack_id" }
      alarm_actions:
        - { get_attr: [scale_in_policy, alarm_url] }

outputs:
  scale_out_url:
    value: { get_attr: [scale_out_policy, alarm_url] }
  scale_in_url:
    value: { get_attr: [scale_in_policy, alarm_url] }
```

8. Required One-Time Environment Fix

```
openstack metric archive-policy-rule create \
--archive-policy-name ceilometer-low-rate \
--metric-pattern "cpu" \
cpu-rate-rule
```

9. Required Per-Deployment Manual Step

```
STACK_ID=$(openstack stack show karimTest -c id -f value)
for VM_ID in $(openstack server list -f value -c ID); do
```

```
openstack metric resource update --type instance \
--attribute server_group:"$STACK_ID" $VM_ID
done
```

10. Verification Sequence

1. `openstack metric archive-policy-rule list` — confirm `cpu-rate-rule` present.
2. `openstack stack create -t autoscale.yaml karimTest` → wait for `CREATE_COMPLETE`.
3. Patch `server_group` on the spawned VM(s) (Section 9).
4. `openstack metric measures show <cpu-uuid> --aggregation rate:mean` — confirm non-empty after one 5-min cycle.
5. `openstack alarm list` — confirm state moves from `insufficient data` to `ok`.
6. Generate real CPU load on a VM console, confirm `cpu_alarm_high` → `alarm` and a new VM is provisioned.
7. Confirm `alarm_url` returns 200 OK with no `Authorization` header on manual `curl` test.
8. Stop load, wait for `cpu_alarm_low` → `alarm`, confirm scale-in deletes a VM back toward `min_size`.

11. Outcome

End-to-end closed-loop autoscaling validated on CirrOS/m1.tiny under DevStack 2024.1: Ceilometer polling → Gnocchi `rate:mean` storage → Aodh threshold evaluation → Heat `alarm_url`-triggered scale-out/scale-in, both directions independently confirmed under real and synthetic CPU load.

12. MLOps: Data Versioning with DVC

- **Objective:** Implement robust data versioning alongside Git to track dataset changes efficiently.
- **Implementation:** Initialized DVC in the repository and tracked a baseline dataset using the classic Iris dataset: `x, y = load_iris(return_X_y=True)`. This approach ensures that data and model states are versioned symmetrically with code commits.

13. Experiment Tracking with MLflow

- **Objective:** Track model hyperparameters, performance metrics, and artifacts to compare different training iterations.
- **Execution:** Trained a `RandomForestClassifier(n_estimators=estimators, max_depth=depth)` on the DVC-tracked dataset.
- **Analysis:** Utilized the `mlflow ui` to visualize the runs. By modifying parameters and comparing every run directly through the web interface, we gained a clear understanding of MLflow's execution logging and visualization capabilities.

14. Model Monitoring: Understanding Data Drift

Data drift is a critical issue in production ML systems that eventually leads to model drift and performance degradation. To address this, a theoretical presentation was created to define the problem, accompanied by a progressive, constructive Jupyter notebook explicitly demonstrating the issue in a real-world scenario.

Synthetic Dataset: The Credit Risk Scenario

We generated realistic synthetic data simulating a **credit default prediction** problem to practically analyze drift.

Features:

- `age` — borrower age (continuous)
- `income` — annual income in USD, log-normal (continuous)
- `credit_score` — FICO score 300–850 (continuous)
- `loan_amount` — requested loan amount (continuous)
- `employ_years` — years at current employer (continuous)
- `region` — geographic region (categorical)

Target: `default` — 1 if the borrower defaulted, 0 otherwise.

Default probability formula (true DGP):

$$P(Y = 1|X) = \sigma \left(-0.02(\text{credit_score} - 680) - 0.8 \ln \frac{\text{income}}{\mu_{inc}} + 0.03 \frac{\text{loan}}{\text{income}} - 0.05 \cdot \text{employ_years} \right)$$

We generated 4 dataset variants to simulate different drift types, analyzed their representations, and compared the specific drift categories:

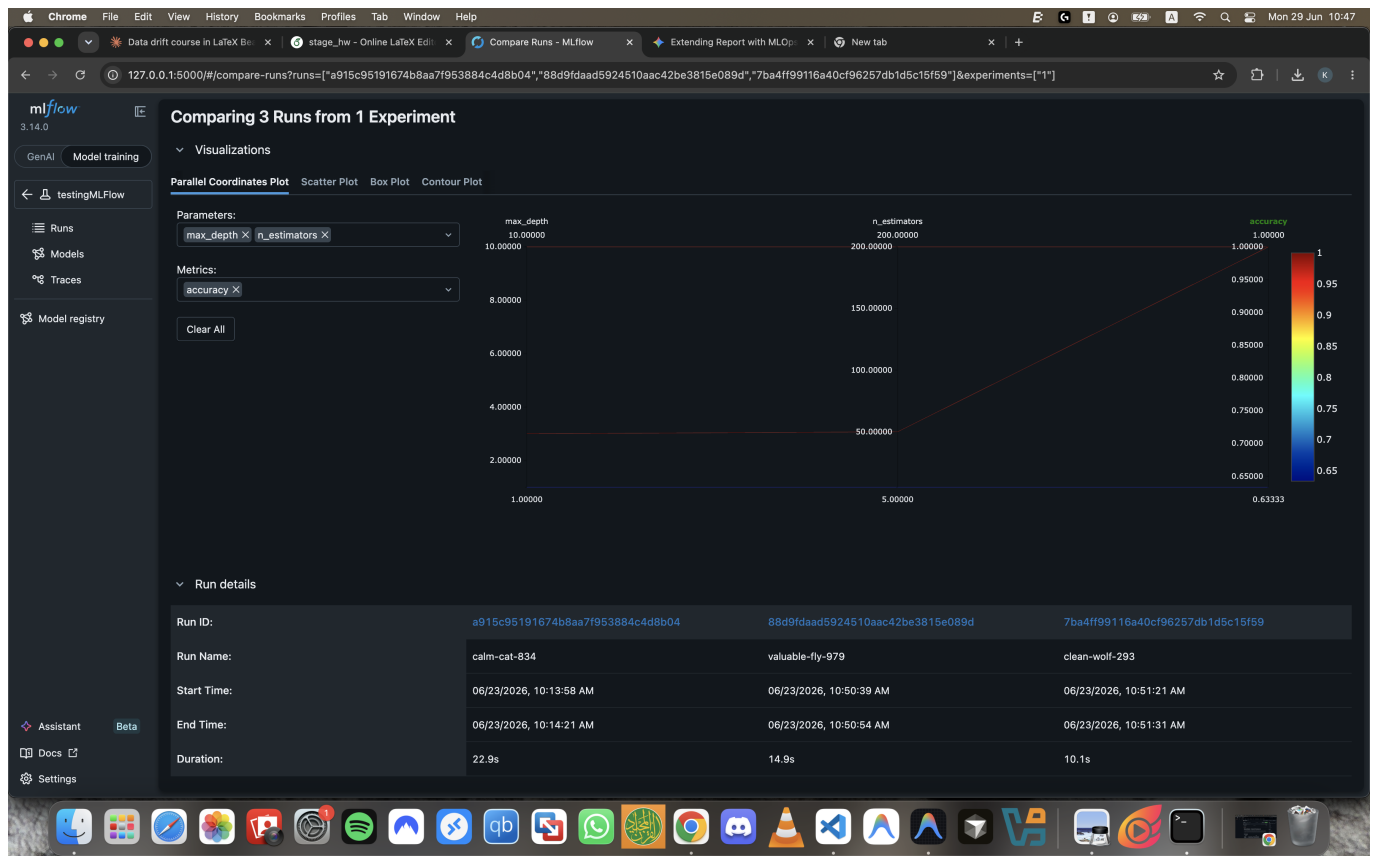


Figure 1: MLflow UI comparing the RandomForest runs

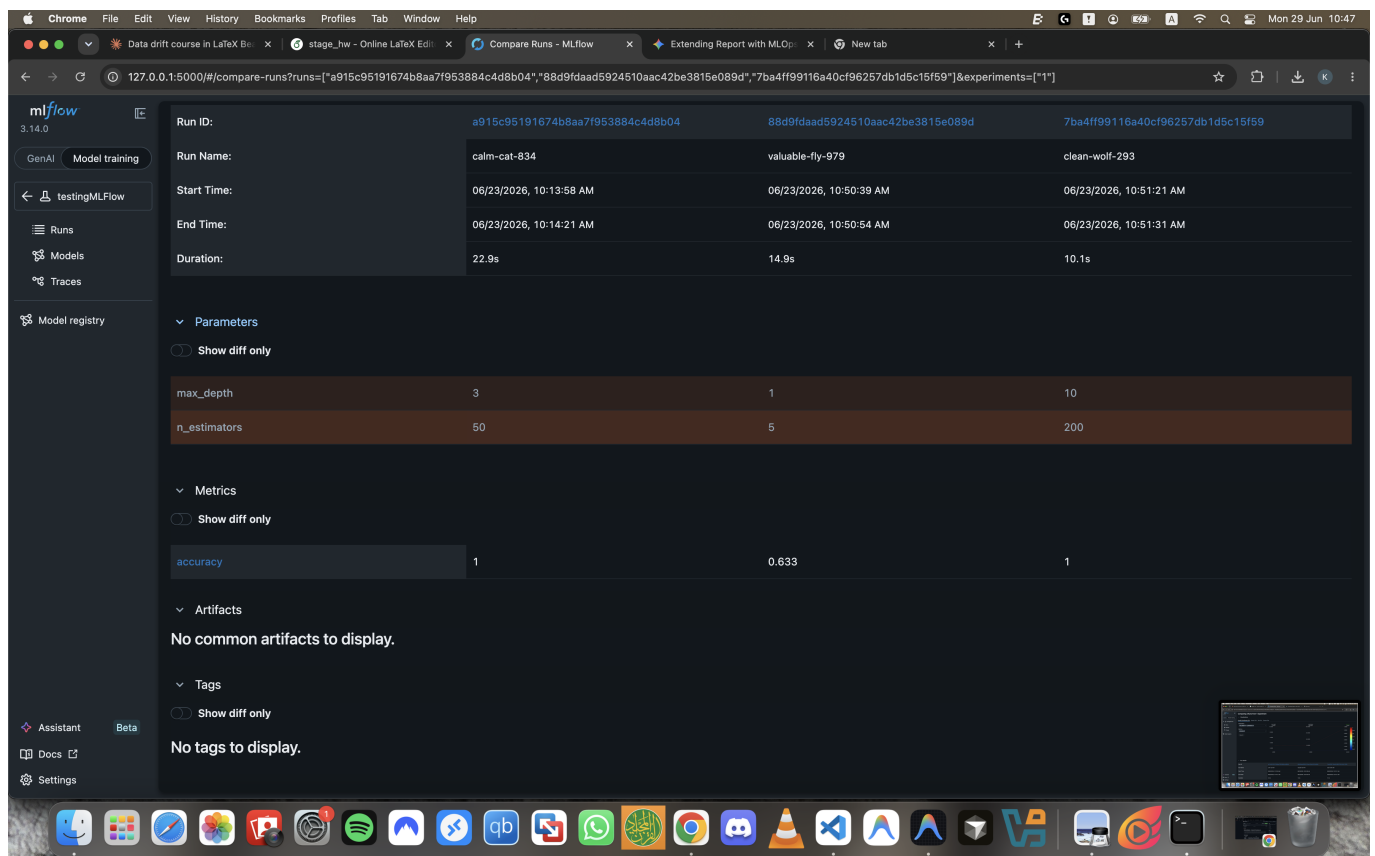


Figure 2: MLflow UI comparing the RandomForest runs

Mode	What changes	What stays same
reference	—	baseline
covariate	$P(X)$	$P(Y X)$
label	$P(Y)$	$P(X Y)$
concept	$P(Y X)$ ⁵	$P(X)$